Endpoint Documentation

Models

UploadDocumentRequest

Used as the request body for both upload endpoints.

```
{
  "operationId": "string | null",
  "id": "string | null",
  "name": "string | null",
  "fileName": "string | null",
  "documentSourceType": "string | null",
  "url": "string | null",
  "content": "string | null"
}
```

Field notes

- `documentSourceType` Expected values in the current implementation are `DIGITAL`, `EXTERNAL_URL`, and `FOLDER`.
- `fileName` Used for DIGITAL uploads when the file is read from local storage.
- `content` Base64-encoded file content for DIGITAL uploads.
- `url` External URL used when `documentSourceType` is `EXTERNAL_URL`.

UploadDocumentResponse

Used as the response envelope for upload success and upload error responses.

```
{
  "operationId": "string | null",
  "success": true,
  "result": {}
}
```

UploadDocumentResult

Used inside `UploadDocumentResponse.result` on upload success.

```
{
  "id": "string",
  "processId": "string",
  "creation_date": "string",
  "modification_date": "string"
}
```

Field notes

- `id` The API returns the created Gestiona entity id. If the upstream create response does not include `id`, the service resolves it from the last path segment of the `self` link in the upstream `links` collection.

UploadDocumentError

Used inside `UploadDocumentResponse.result` on upload errors and download errors.

```
{
  "code": 400,
  "name": "Bad Request",
  "kind": "Validation",
  "message": "string"
}
```

Possible `kind` values for upload

- `Configuration`
- `Validation`
- `NotFound`
- `Upstream`

Possible `kind` values for download

- `Configuration`
- `Validation`
- `NotFound`
- `Upstream`

Download Success Output

The download endpoint does not return JSON on success. It returns the raw document bytes in the response body.

Relevant response characteristics

- Body: binary file content
- `Content-Type`: document MIME type returned by Gestiona, or `application/octet-stream` as fallback
- `Content-Disposition`: attachment, with automatic download filename

The underlying DLL model used by the service layer is

```
{
  "documentId": "string",
  "fileName": "string | null",
  "contentType": "string | null",
  "storageSize": 0,
  "storageExtension": "string | null",
  "storageMimeType": "string | null",
  "storageMd5": "string | null",
  "storageSha1": "string | null",
  "storageSha512": "string | null",
  "content": "byte[]"
}
```

Endpoints

1. POST `/processes/documents`

Creates a document by resolving the target Gestiona process_id (file id in gestiona) from the query parameter `process_number`.

Query parameters

- `process_number` required

Request body model

- `UploadDocumentRequest`

Success response

- HTTP `200 OK`
- Body model: `UploadDocumentResponse`
- `result` shape: `UploadDocumentResult`

Success example

```
{
  "operationId": "op-123",
  "success": true,
  "result": {
    "id": "document-id",
    "processId": "file-id",
    "creation_date": "2026-05-08 10:00:00",
    "modification_date": "2026-05-08 10:00:00"
  }
}
```

Error response

- HTTP `400`, `404`, `500`, or propagated upstream status code
- Body model: `UploadDocumentResponse`
- `result` shape: `UploadDocumentError`

Error example

```
{
  "operationId": "op-123",
  "success": false,
  "result": {
    "code": 400,
    "name": "Bad Request",
    "kind": "Validation",
    "message": "process_number query parameter is required."
  }
}
```

2. POST `/processes/{process_id}/documents`

Creates a document directly in the Gestiona file identified by `process_id`.

Route parameters

- `process_id` required

Request body model

- `UploadDocumentRequest`

Success response

- HTTP `200 OK`
- Body model: `UploadDocumentResponse`
- `result` shape: `UploadDocumentResult`

Error response

- HTTP `400`, `404`, `500`, or propagated upstream status code
- Body model: `UploadDocumentResponse`
- `result` shape: `UploadDocumentError`

Notes

- For `DIGITAL` uploads, either `fileName` or `content` must be provided.
- If both `fileName` and `content` are provided, the current implementation uses `content`.
- On successful create operations, `result.id` may come either from the upstream `id` field or, when that field is missing, from the last segment of the upstream `self` link.

3. POST `/processes/documents/{folder_id}`

Creates a document inside the Gestiona folder identified by `folder_id`, after resolving the target Gestiona file from the query parameter `process_number`.

Route parameters

- `folder_id` required

Query parameters

- `process_number` required

Request body model

- `UploadDocumentRequest`

Success response

- HTTP `200 OK`
- Body model: `UploadDocumentResponse`
- `result` shape: `UploadDocumentResult`

Error response

- HTTP `400`, `404`, `500`, or propagated upstream status code
- Body model: `UploadDocumentResponse`
- `result` shape: `UploadDocumentError`

Notes

- This route uses the same document creation flows as the process-number route, but targets the upstream Gestiona endpoint with the folder id in the last path segment.
- For `DIGITAL` uploads, either `fileName` or `content` must be provided.
- If both `fileName` and `content` are provided, the current implementation uses `content`.
- On successful create operations, `result.id` may come either from the upstream `id` field or, when that field is missing, from the last segment of the upstream `self` link.

4. POST `/processes/{process_id}/documents/{folder_id}`

Creates a document directly inside the Gestiona folder identified by `folder_id`, under the file identified by `process_id`.

Route parameters

- `process_id` required
- `folder_id` required

Request body model

- `UploadDocumentRequest`

Success response

- HTTP `200 OK`
- Body model: `UploadDocumentResponse`
- `result` shape: `UploadDocumentResult`

Error response

- HTTP `400`, `404`, `500`, or propagated upstream status code
- Body model: `UploadDocumentResponse`
- `result` shape: `UploadDocumentError`

Notes

- This route uses the same document creation flows as the file-level route, but targets the upstream Gestiona endpoint with the folder id in the last path segment.
- For `DIGITAL` uploads, either `fileName` or `content` must be provided.
- If both `fileName` and `content` are provided, the current implementation uses `content`.
- On successful create operations, `result.id` may come either from the upstream `id` field or, when that field is missing, from the last segment of the upstream `self` link.

5. GET `/documents/{document_id}`

Downloads a document from Gestiona. This is an absolute route and is not prefixed by `/processes`.

Route parameters

- `document_id` required

Query parameters

- `operationId` optional

Request body model

- none

Success response

- HTTP `200 OK`
- Body: raw binary document content
- Headers: - `Content-Type`: document MIME type returned by Gestiona, or `application/octet-stream` as fallback - `Content-Disposition: attachment; filename=...` - `X-Operation-Id`: present when `operationId` is provided in the request - `X-Storage-Extension`: present when the upstream document metadata includes a storage extension

Download filename resolution

The controller chooses the download filename in this order:

1. `document.fileName`
2. `document.documentId + "." + document.storageExtension`
3. `document.documentId`

Validation behavior

- If `document_id` is empty or whitespace, the endpoint returns HTTP `400`
- If Postman sends an unresolved variable such as `{{document_id}}`, the endpoint returns HTTP `400`

Error response

- HTTP `400`, `404`, `500`, or propagated upstream status code
- Body model: `UploadDocumentResponse`
- `result` shape: `UploadDocumentError`

Error example

```
{
  "operationId": "op-123",
  "success": false,
  "result": {
    "code": 404,
    "name": "Not Found",
    "kind": "NotFound",
    "message": "Failed to download document from Gestiona: 12345."
  }
}
```

Notes

- Successful downloads do not return JSON
- Download errors reuse the same `UploadDocumentResponse` envelope as upload errors
- When `operationId` is provided in the download request, it is echoed back: - in the error JSON body on failure - in the `X-Operation-Id` response header on success
- When the upstream response includes document storage extension metadata, it is exposed in the `X-Storage-Extension` response header